



Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning

Chenkai Weng, Northwestern University; Kang Yang, State Key Laboratory of Cryptology; Xiang Xie, Shanghai Key Laboratory of Privacy-Preserving Computation and MatrixElements Technologies; Jonathan Katz, University of Maryland; Xiao Wang, Northwestern University

<https://www.usenix.org/conference/usenixsecurity21/presentation/weng>

**This paper is included in the Proceedings of the
30th USENIX Security Symposium.**

August 11-13, 2021

978-1-939133-24-3

**Open access to the Proceedings of the
30th USENIX Security Symposium
is sponsored by USENIX.**

Mystique: Efficient Conversions for Zero-Knowledge Proofs with Applications to Machine Learning

Chenkai Weng
Northwestern University

Kang Yang (✉)
State Key Laboratory of Cryptology

Xiang Xie
Shanghai Key Laboratory of
Privacy-Preserving Computation and
MatrixElements Technologies

Jonathan Katz
University of Maryland

Xiao Wang (✉)
Northwestern University

Abstract

Recent progress in interactive zero-knowledge (ZK) proofs has improved the efficiency of proving large-scale computations significantly. Nevertheless, real-life applications (e.g., in the context of private inference using deep neural networks) often involve highly complex computations, and existing ZK protocols lack the expressiveness and scalability to prove results about such computations efficiently.

In this paper, we design, develop, and evaluate a ZK system (Mystique) that allows for efficient conversions between arithmetic and Boolean values, between publicly committed and privately authenticated values, and between fixed-point and floating-point numbers. Targeting large-scale neural-network inference, we also present an improved ZK protocol for matrix multiplication that yields a $7\times$ improvement compared to the state-of-the-art. Finally, we incorporate Mystique in Rosetta, a TensorFlow-based privacy-preserving framework.

Mystique is able to prove correctness of an inference on a private image using a committed (private) ResNet-101 model in 28 minutes, and can do the same task when the model is public in 5 minutes, with only a 0.02% decrease in accuracy compared to a non-ZK execution when testing on the CIFAR-10 dataset. Our system is the first to support ZK proofs about neural-network models with over 100 layers with virtually no loss of accuracy.

1 Introduction

Zero-knowledge (ZK) proofs allow one party with a secret witness to prove some statement about that witness without revealing any additional information. In recent years we have seen massive progress in the efficiency and scalability of ZK proofs based on many different ideas [14, 17, 34, 38, 39]. With such improvements, we envision a huge potential in applying ZK proofs to *machine learning* (ML) applications, particularly neural-network inference. As examples:

- **Zero-knowledge proofs of evasion attacks.** A pre-trained model $\mathcal{M}$ might be publicly released to be used by the

general public. Using a ZK protocol, a white-hat hacker who discovers a bug in the model (e.g., an evasion attack) could prove existence of that bug in zero knowledge, e.g., they could prove knowledge of two “close” inputs x_1 and x_2 for which $\mathcal{M}(x_1) \neq \mathcal{M}(x_2)$.

- **Zero-knowledge proofs of correct inference.** An ML model may require huge effort to train and thus may only be accessible as a paid service (e.g., GPT-3 that contains 175 billion parameters [19]). In this case, the model parameters are kept private, and users need to send their inputs to the owner of an ML model to be classified. Currently, such users have no guarantee that the model owner applies the correct model. Using a ZK protocol, the model owner could publicly commit to a model, and then proves in zero knowledge that the committed model was applied to the user’s submitted input, yielding the claimed result.
- **Zero-knowledge proofs of private benchmarks.** An ML model may be evaluated on private testing data. Here, the owner of the testing data can publicly commit to its data; a model trainer can then send its model (developed using independent training data) to the data owner, who locally evaluates the accuracy of the model. The data owner can use a ZK protocol to prove that the submitted model was executed on the committed data.

Unfortunately, after examining existing ZK proof systems, we found that no existing solutions were sufficiently scalable or practical for any of the above applications once reasonably complicated neural-network models were involved. For example, zk-SNARKs [9, 10, 12, 16, 20, 41, 47, 50] provide excellent proof size and verification time, and support verifiable computation on committed data [29]. However, state-of-the-art zk-SNARKs require the memory of the prover to be proportional to the statement size; proving a statement with a billion constraints would require about 640 GB of memory. For ML applications, they can only handle simple models like decision trees [57]. Recent ZK protocols based on *subfield vector oblivious linear evaluation* (sVOLE) [7, 27, 52, 54],

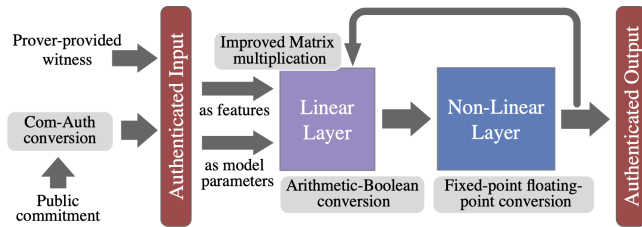


Figure 1: Overview of our system for ZK neural-network inference.

privacy-free garbled circuits [30, 37, 39], or the “MPC-in-the-head” paradigm [4, 8, 21, 33, 40] are efficient in terms of execution time and memory overhead, but do not work efficiently with publicly committed data and the overall communication is still fairly large. While in principle one could use zk-SNARKs with recursive composition [15, 23], their concrete performance is still quite poor.

1.1 Our Contributions

We propose a system (Mystique¹) based on recent sVOLE-based interactive ZK protocols that includes a set of building blocks for efficient ZK proofs of large-scale neural-network inference. Crucially, our system includes efficient techniques for three types of conversions:

1. **Arithmetic/Boolean values.** Inspired by similar ideas in the setting of secure computation [28], we design optimized protocols to efficiently convert between arithmetic and Boolean values (to support mixed-mode circuits) in the context of sVOLE-based zero knowledge.
2. **Committed/authenticated values.** To allow publicly committed values to be used in ZK proofs, we design an efficient protocol that converts such values to values that are privately authenticated by the prover and verifier, and can thus be used directly in sVOLE-based ZK protocols.
3. **Fixed-point/floating-point values.** We designed circuits for IEEE-754-compliant floating-point operations, and designed efficient protocols to convert between fixed-point values (encoded in a field) and floating-point numbers.

In addition to the above, we also design an efficient ZK proof for matrix multiplication, such that the number of private multiplications required is *sublinear* in the matrix size. Compared to the previously best-known ZK proof for matrix multiplication [54], our ZK protocol improves the execution time by a factor of $7\times$.

We integrated the above in Rosetta [22], a privacy-preserving framework based on TensorFlow [1], and use the

¹ Mystique is a shape-shifter; our system supports efficient conversions (“shape shifting”) in zero knowledge.

resulting system for ZK proofs regarding neural-network inference. As shown in Figure 1, linear layers of the neural network are accelerated by using our matrix-multiplication optimization, while the non-linear layers rely on our fixed-point/floating-point conversions. (We implemented ReLU, Sigmoid, Max Pooling, SoftMax, and Batch Normalization in the non-linear layers; other operations can be added easily.) All computations can be done using either arithmetic or Boolean values, depending on which is more efficient at any given step. Due to our improved cryptographic protocols and integrated implementation, we can implement ZK proofs on large neural networks (e.g., ResNet-50 and ResNet-101) with millions of model parameters; see Section 7.

2 Preliminaries

2.1 Notation

We use λ and ρ to denote the computational and statistical security parameters, respectively. For a finite set S , we use $x \leftarrow S$ to denote that x is chosen uniformly from S . For $a, b \in \mathbb{N}$, we denote by $[a, b]$ the set $\{a, \dots, b\}$, and by $[a, b)$ the set $\{a, \dots, b-1\}$. We use bold lower-case letters like $\mathbf{x}$ for column vectors, and denote by x_i the i -th component of $\mathbf{x}$ where x_1 is the first entry. We use $\mathbf{x}^\top$ to denote the transposition of $\mathbf{x}$. We use $\text{negl}(\cdot)$ to denote a **negligible function**.

For an extension field $\mathbb{F}_{q^k}$ of a field $\mathbb{F}_q$, we fix some **monic**, irreducible polynomial $f(X)$ of degree k so that $\mathbb{F}_{q^k} \cong \mathbb{F}_q[X]/f(X)$. Thus, every element $a \in \mathbb{F}_{q^k}$ can be uniquely written as $a = \sum_{h \in [1, k]} a_h \cdot X^{h-1}$ with $a_h \in \mathbb{F}_q$ for all $h \in [1, k]$. When we write arithmetic expressions involving both elements of $\mathbb{F}_q$ and $\mathbb{F}_{q^k}$, operations are performed in $\mathbb{F}_{q^k}$ with elements of $\mathbb{F}_q$ viewed as elements of $\mathbb{F}_{q^k}$ in the natural way. In general, we work in an extension field such that $q^k \geq 2^\rho$.

2.2 Universal Composability

We say that protocol Π *UC-realizes* ideal functionality $\mathcal{F}$ if for any *probabilistic polynomial time* (PPT) adversary $\mathcal{A}$, there exists a PPT simulator $\mathcal{S}$ such that for any PPT environment $\mathcal{Z}$ with arbitrary auxiliary input, the output distribution of $\mathcal{Z}$ in the *real-world* execution where $\mathcal{Z}$ interacts with $\mathcal{A}$ and the parties running Π is computationally indistinguishable from the output distribution of $\mathcal{Z}$ in the *ideal-world* execution where $\mathcal{Z}$ interacts with $\mathcal{S}$ and $\mathcal{F}$. In the $\mathcal{G}$ -hybrid model the parties execute a protocol given access to ideal functionality $\mathcal{G}$. We say that protocol Π UC-realizes $\mathcal{F}$ in the $\mathcal{G}$ -hybrid model with statistical error ϵ if the statistical difference between the output distributions of $\mathcal{Z}$ in the real-world execution and hybrid-world execution is bounded by ϵ .

2.3 Information-theoretic MACs

We use *information-theoretic message authentication codes* (IT-MACs), which were originally proposed for maliciously secure two-party computation [13, 43]. We authenticate values in $\mathbb{F}_q$, but the authentication itself is done over an extension field $\mathbb{F}_{q^k}$. Specifically, let $\Delta \in \mathbb{F}_{q^k}$ be a uniform *global key* known only to the verifier $\mathcal{V}$. A value $x \in \mathbb{F}_q$ known by the prover $\mathcal{P}$ is authenticated by giving $\mathcal{V}$ a uniform *local key* $K \in \mathbb{F}_{q^k}$ and giving $\mathcal{P}$ the corresponding tag

$$M = K + \Delta \cdot x \in \mathbb{F}_{q^k}.$$

We denote such an authenticated value by $[x] = (x, M, K)$, meaning that $\mathcal{P}$ holds (x, M) and $\mathcal{V}$ holds K . When we want to make the field explicit, we write $[x]_q$. We extend the above notation to vectors or matrices of authenticated values as well. For example, $[x]$ means that $\mathcal{P}$ holds $\mathbf{x} \in \mathbb{F}_q^n$ and $\mathbf{M} \in (\mathbb{F}_{q^k})^n$, while $\mathcal{V}$ holds $\mathbf{K} \in (\mathbb{F}_{q^k})^n$ with $\mathbf{M} = \mathbf{K} + \Delta \cdot \mathbf{x}$.

Authenticated values are additively homomorphic. That is, given authenticated values $[x_1], \dots, [x_\ell]$ and public coefficients $c_1, \dots, c_\ell, c \in \mathbb{F}_q$, the parties can compute $[y] = \sum_{i=1}^\ell c_i \cdot [x_i] + c$ using only local computation.

Batch opening. An authenticated value $[x]$ can be opened by having $\mathcal{P}$ send (x, M) to $\mathcal{V}$, who verifies $M = K + \Delta \cdot x$. When opening ℓ values, it is possible to do better than ℓ parallel repetitions of this procedure; specifically, all ℓ values can be opened using only $\ell \log q + \lambda$ bits of communication. We use BatchCheck for the following batch-opening procedure:

- Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda$ be a hash function modeled as a random oracle. Suppose that $[x_1], \dots, [x_\ell]$ are ℓ authenticated values to be opened.
- $\mathcal{P}$ sends $x_1, \dots, x_\ell \in \mathbb{F}_q$ to $\mathcal{V}$.
- Additionally:
 - If $q = 2$ and $k = \lambda$, the two parties compute $\chi := H(x_1, \dots, x_\ell) \in \mathbb{F}_{2^\lambda}$. $\mathcal{P}$ computes $\sigma := \sum_{i \in [1, \ell]} M_i \cdot \chi^i \in \mathbb{F}_{2^\lambda}$ and sends it to $\mathcal{V}$, who checks whether $\sigma = \sum_{i \in [1, \ell]} (K_i + \Delta \cdot x_i) \cdot \chi^i$. As in prior work [35, 52], the soundness error is bounded by $(q_H + \ell + 1)/2^\lambda = \text{negl}(\lambda)$, where q_H is the number of queries to H .
 - Otherwise, $\mathcal{P}$ computes $\sigma := H(M_1, \dots, M_\ell) \in \{0, 1\}^\lambda$ and sends it to $\mathcal{V}$, who can then check whether $\sigma := H(K_1 + \Delta \cdot x_1, \dots, K_\ell + \Delta \cdot x_\ell)$. The soundness error is at most $1/p^k + q_H/2^\lambda = 1/p^k + \text{negl}(\lambda)$ [26].

When the opened values are all zero and so need not be sent, we use CheckZero to denote the batch-opening procedure.

Authenticated values from sVOLE. We can view authenticated values as *subfield vector oblivious linear evaluation* (sVOLE) correlations. Thus, random authenticated values can be efficiently generated using the recent LPN-based sVOLE protocols [18, 46, 52, 55], which have communication complexity *sublinear* in the number of authenticated values.

2.4 Zero-Knowledge Proofs based on sVOLE

Several recent works [7, 27, 52, 54] explored the efficiency of sVOLE-based interactive ZK proofs. One can consider authenticated values as a form of commitments on values held by a prover $\mathcal{P}$, which can be verified by the verifier $\mathcal{V}$. Therefore, one can construct a ZK protocol following the “commit-and-prove” paradigm as follows:

1. In a preprocessing phase, $\mathcal{P}$ and $\mathcal{V}$ execute the sVOLE protocol to generate $n + N$ uniform authenticated values, where n is the witness size and N is the number of multiplication gates in the circuit. The parties also compute a uniform authenticated value $A_1^* \in \mathbb{F}_{q^k}$ by generating k uniform authenticated values in $\mathbb{F}_q$ and then using packing (see [54] for details). Thus, $\mathcal{P}$ obtains $A_0^*, A_1^* \in \mathbb{F}_{q^k}$ and $\mathcal{V}$ gets $B^* \in \mathbb{F}_{q^k}$ such that $B^* = A_0^* + \Delta \cdot A_1^*$.
2. Using the uniform authenticated values, $\mathcal{P}$ commits to all the wire values in an evaluation of the circuit on its witness. In particular, for each input wire of the circuit or output wire of a multiplication gate with associated value $x \in \mathbb{F}_q$, $\mathcal{P}$ sends $d = x - r \in \mathbb{F}_q$ to $\mathcal{V}$ and then both parties compute $[x] := [r] + d$, where $[r]$ is a random authenticated value computed in the previous step. Due to the additively homomorphic property of the underlying authenticated values, the parties can process addition gates for free and so this allows $\mathcal{P}$ and $\mathcal{V}$ to compute authenticated values on every wire in the circuit.
3. $\mathcal{P}$ proves that the committed values at all multiplication gates are correct by running a consistency-check procedure with $\mathcal{V}$. The known sVOLE-based ZK proofs [7, 27, 52, 54] use different consistency-check procedures. The state-of-the-art consistency check [27, 54] works as follows:

- (a) Consider the i -th multiplication gate with authenticated values $([x], [y], [z])$. If it is computed correctly (i.e., $z = x \cdot y$), then:

$$\begin{aligned} \overbrace{B_i = K_x \cdot K_y + K_z \cdot \Delta}^{\text{known to } \mathcal{V}} &= (M_x - x \cdot \Delta) \cdot (M_y - y \cdot \Delta) + (M_z - z \cdot \Delta) \cdot \Delta \\ &= M_x \cdot M_y + (M_z - y \cdot M_x - x \cdot M_y) \cdot \Delta + (x \cdot y - z) \cdot \Delta^2 \\ &= \underbrace{M_x \cdot M_y}_{\substack{\text{known to } \mathcal{P} \\ \text{denoted by } A_{0,i}}} + \underbrace{(M_z - y \cdot M_x - x \cdot M_y)}_{\substack{\text{known to } \mathcal{P} \\ \text{denoted by } A_{1,i}}} \cdot \underbrace{\Delta}_{\substack{\text{known to } \mathcal{V} \\ \text{global key}}}. \end{aligned}$$

If $z \neq x \cdot y$, then the above holds with probability at most $2/q^k$ over choice of Δ .

- (b) The parties can check all N of the above equations at once by taking a random linear combination. In particular, $\mathcal{V}$ samples and sends a uniform $\chi \in \mathbb{F}_{q^k}$ to $\mathcal{P}$. Then $\mathcal{P}$ sends $U_0 := \sum_{i \in [1, N]} A_{0,i} \cdot \chi^i + A_0^*$ and

$U_1 := \sum_{i \in [1, N]} A_{1,i} \cdot \chi^i + A_1^*$ to $\mathcal{V}$, who checks that $\sum_{i \in [1, N]} B_i \cdot \chi^i + B^* = U_0 + U_1 \cdot \Delta$. This can be made non-interactive using the Fiat-Shamir transform (i.e., computing χ by hashing the protocol transcript), and can be further optimized when q is large [52, 54].

The ideal functionality for ZK proofs in this setting is summarized in Figure 2, where both arithmetic and Boolean circuits are considered. (Prior work [7, 27, 52, 54] efficiently realizes either arithmetic or Boolean circuits, but not mixed-mode computations.) For convenience, we also include in the ideal functionality the other conversions we support in our work (see Figure 3).

3 Technical Overview

As mentioned in Section 1.1, we propose new protocols for arithmetic-Boolean and commitment-authentication conversions that are highly useful in real-world applications. We summarize in Figure 3 for the functionality definition of the two types of conversions. At a high level, our arithmetic-Boolean conversion allows authenticated values to be converted between arithmetic and Boolean circuits, while at the same time ensuring that the consistent values are converted. The commitment-authentication conversion allows us to convert from publicly committed values to privately authenticated values: the former provides a unified view of data across multiple verifiers, while the later can be efficiently processed by the sVOLE-based ZK protocols.

For ML applications, we also present the conversion between fixed-point and floating-point numbers, and an improved ZK proof for matrix multiplication. Below we provide an overview of our techniques and leave the detailed protocol description as well as proofs of security in later sections.

3.1 Arithmetic-Boolean Conversion

Enabling ZK proofs to support both arithmetic and Boolean circuits have been an important topic and studied in prior work. Particularly, in zk-SNARKs, it is often referred to as bit-decomposition [11, 12, 24, 44, 49]. Suppose that a prover has a witness x and the statement needs to compute on the bit representation of x . The prover can provide a bit decomposition of x , namely $\{x_i\}_{i \in [0, m]}$, with m as the bit-length of x . The prover can then prove in zero-knowledge that $x_i \cdot (x_i - 1) = 0$ for all $i \in [0, m]$ and $\sum_{i \in [0, m]} x_i \cdot 2^i = x$. Essentially, this is a way to simulate bit computation on an arithmetic circuit, which does not improve the underlying ZK proof.

Another solution [36] is to combine garbled-circuit zero-knowledge proofs [30, 37, 39] (GCZK) with arithmetic garbling [6]. However, it only supports multiplication by public constants, and thus proving multiplication of two values over field $\mathbb{F}_p$ still needs to take communication of $\lambda \log p$ bits.

Functionality $\mathcal{F}_{\text{authZK}}$

This functionality is parameterized by a prime $p > 2$ and an integer k such that $p^k \geq 2^p$, and can invoke a macro $\text{Auth}()$ defined in Figure 4. Let $m = \lceil \log p \rceil$.

Initialize: On input (init) from a prover $\mathcal{P}$ and verifier $\mathcal{V}$, sample $\Delta \leftarrow \mathbb{F}_{2^\lambda}$ and $\Gamma \leftarrow \mathbb{F}_{p^k}$ if $\mathcal{V}$ is honest, and receive $\Delta \in \mathbb{F}_{2^\lambda}$ and $\Gamma \in \mathbb{F}_{p^k}$ from the adversary otherwise. Store (Δ, Γ) and send them to $\mathcal{V}$, and ignore all subsequent (init) commands.

Input: On input (input, id, w, q) from $\mathcal{P}$ and (input, id, q) from $\mathcal{V}$, where id is a fresh identifier, $w \in \mathbb{F}_q$ and $q \in \{2, p\}$, execute $\text{Auth}(w, q)$ so that the parties obtain $[w]$, and store (id, $q, [w]$).

Output: On input (output, id) from two parties where id is present in memory, retrieve (id, $q, [z]$) and output $z \in \mathbb{F}_q$ to $\mathcal{V}$.

Circuit-based commands

Random: On input (random, id, q) from $\mathcal{P}$ and $\mathcal{V}$ with id a fresh identifier and $q \in \{2, p\}$, sample $w \leftarrow \mathbb{F}_q$ if $\mathcal{P}$ is honest; otherwise receive $w \in \mathbb{F}_q$ from the adversary. Execute $\text{Auth}(w, q)$ so that the parties obtain $[w]$, and store (id, $q, [w]$).

Linear combination: On input (lincomb, id, $\{\text{id}_i\}_{i \in [1, \ell]}$, $\{c_i\}_{i \in [0, \ell]}$, q) from two parties, where (id _{i} , q) is present in memory for $i \in [1, \ell]$ and $c_i \in \mathbb{F}_q$ for $i \in [0, \ell]$, retrieve (id _{i} , $q, [x_i]$) for all $i \in [1, \ell]$, and then compute $[y] := c_0 + \sum_{i \in [1, \ell]} c_i \cdot [x_i]$, store (id, $q, [y]$) and output $[y]$ to the parties.

Multiply: On input (mult, id, id₁, id₂, q) from both parties, where (id₁, q) and (id₂, q) are present in memory, retrieve (id₁, $q, [x]$) and (id₂, $q, [y]$). Then compute $z := x \cdot y \in \mathbb{F}_q$, run $\text{Auth}(z, q)$ so that the parties obtain $[z]$, and store (id, $q, [z]$).

Figure 2: Zero-knowledge functionality with authenticated values.

Our approach. Some recent works on sVOLE-based ZK protocols achieve high concrete efficiency [7, 27, 52, 54]. They support either arithmetic or Boolean circuits, and compute a circuit with authenticated wire values. The conversion between two types of circuits boils down to converting between authenticated arithmetic values and authenticated Boolean values. These cases are similar to some *secure multi-party computation* (MPC) protocols, which only support operations over either arithmetic or Boolean circuits, and use IT-MACs to authenticate secretly-sharing values.

In the MPC setting, converting authenticated shares between arithmetic and Boolean circuits can be accomplished

Functionality $\mathcal{F}_{\text{authZK}}$, continued

Conversion between arithmetic and Boolean values

From arithmetic to Boolean: On input (convertA2B, $\text{id}, \text{id}_0, \dots, \text{id}_{m-1}$) from $\mathcal{P}$ and $\mathcal{V}$ where (id, p) is present in memory, retrieve $(\text{id}, p, [x]_p)$ and decompose $x \in \mathbb{F}_p$ as $(x_0, \dots, x_{m-1}) \in \{0, 1\}^m$ such that $x = \sum_{i=0}^{m-1} x_i \cdot 2^i \bmod p$. Then, for $i \in [0, m)$, execute $\text{Auth}(x_i, 2)$ so that the parties obtain $[x_i]_2$ and store $(\text{id}_i, 2, [x_i]_2)$.

From Boolean to arithmetic: On input (convertB2A, $\text{id}_0, \dots, \text{id}_{m-1}, \text{id}$) from two parties, where $(\text{id}_i, 2)$ is present in memory for $i \in [0, m)$, retrieve $(\text{id}_i, 2, [x_i]_2)$ for each $i \in [0, m)$. Then, compute $x := \sum_{i=0}^{m-1} x_i \cdot 2^i \bmod p$, execute $\text{Auth}(x, p)$ so that the parties obtain $[x]_p$, and store $(\text{id}, p, [x]_p)$.

Conversion from publicly committed values to privately authenticated values

Commit: On input (commit, cid, x, q) from $\mathcal{P}$ with cid a fresh identifier, $x \in \mathbb{F}_q$ and $q \in \{2, p\}$, store (cid, q, x) and send (cid, q) to multiple potential verifiers.

From committed to authenticated values: On input (convertC2A, cid, id) from $\mathcal{P}$ and a verifier $\mathcal{V}$, where cid is present in memory and id is a fresh identifier, retrieve (cid, q, x) , and then execute $\text{Auth}(x, q)$ so that the two parties obtain $[x]$ and store $(\text{id}, q, [x])$.

Figure 3: Zero-knowledge functionality with authenticated values, continued.

Macro $\text{Auth}(x, q)$

On input $x \in \mathbb{F}_q$ and $q \in \{2, p\}$, this subroutine interacts with two parties $\mathcal{P}$ and $\mathcal{V}$, and generates an authenticated value $[x]$ for the parties. Let $k = \lambda$ and $\Phi = \Delta$ if $q = 2$. Let $k \in \mathbb{N}$ such that $q^k \geq 2^p$ and $\Phi = \Gamma$ if $q = p$.

1. If $\mathcal{V}$ is honest, sample $K \leftarrow \mathbb{F}_{q^k}$. Otherwise, receive $K \in \mathbb{F}_{q^k}$ from the adversary.
2. If $\mathcal{P}$ is honest, compute $M := K + \Phi \cdot x \in \mathbb{F}_{q^k}$. Otherwise, receive $M \in \mathbb{F}_{q^k}$ from the adversary and recompute $K := M - \Phi \cdot x \in \mathbb{F}_{q^k}$.
3. Output $[x]$ to the parties, i.e., send (x, M) to $\mathcal{P}$ and K to $\mathcal{V}$.

Figure 4: Macro used by functionalities $\mathcal{F}_{\text{authZK}}$ and $\mathcal{F}_{\text{zk-edaBits}}$ to generate authenticated values.

by so-called *doubly authenticated bits* (daBits) [3, 25, 45]. The key idea of daBits is to prepare for secretly-shared random bits that are authenticated in both fields $\mathbb{F}_2$ and $\mathbb{F}_p$ with a large

prime p (meaning that $p \geq 2^p$), so that one set of MAC tags support Boolean operations (i.e., AND and XOR), while the other set of MAC tags are arithmetic-operation (i.e., MULT and ADD) homomorphic. To perform a conversion, we need $m = \lceil \log p \rceil$ such daBits, which are used to convert the shares of $x_0, \dots, x_{m-1} \in \mathbb{F}_2$ to that of $x = \sum_{h=0}^{m-1} x_h \cdot 2^h \in \mathbb{F}_p$, where the related MAC tags are also converted accordingly. In the ZK setting, we can use a similar method. Unfortunately, although we can authenticate a field element over $\mathbb{F}_p$ efficiently in communication of $O(\log p)$ bits, authenticating a bit with the MAC tag in $\mathbb{F}_p$ still takes $O(\log p)$ bits (instead of one bit) for communication. As a result, the conversion requires a total communication of $O(\log^2 p)$ bits for generating m daBits, not even counting the cost to perform conversion using daBits.

To overcome this, we instead follow the more recent *extended daBits* (edaBits) [28], which can be viewed as a more compact representation of daBits. An edaBit consists of a set of m random secretly-shared bits $([r_0]_2, \dots, [r_{m-1}]_2)$ with the MAC tags in $\mathbb{F}_{2^\lambda}$ and a secretly-shared field element $[r]_p$ with the MAC tag in $\mathbb{F}_p$, such that $(r_0, \dots, r_{m-1}) \in \mathbb{F}_2^m$ is equal to the bit-decomposition of $r \in \mathbb{F}_p$. Such an edaBit is sufficient to perform conversion between arithmetic and Boolean circuits. Now, generating an edaBit requires only $O(\log p)$ bits of communication. Inspired by the edaBits approach for MPC, we first construct *ZK-friendly edaBits* (zk-edaBits) to keep compatible with the recent sVOLE-based ZK proofs [7, 27, 52, 54], and then construct two conversion protocols using our random zk-edaBits, where one can convert authenticated values from arithmetic to Boolean circuits and the other converts in another direction.

Constructing zk-edaBits. Similar to edaBit in the MPC setting, a zk-edaBit consists of a random authenticated value $[r]_p$ and m authenticated bits $[r_0]_2, \dots, [r_{m-1}]_2$, such that $r = \sum_{h=0}^{m-1} r_h \cdot 2^h \bmod p$. In the ZK setting, the prover is allowed to know $(r_0, \dots, r_{m-1}, r)$ as it knows all wire values in the circuit, and thus the secret sharing of these values is *unnecessary*. Here, we do not assume that p is a large prime, and instead allow any prime $p > 2$, as we consider authentication is done in an extension field $\mathbb{F}_{p^k}$ with $p^k \geq 2^p$.

Using two sVOLE-based ZK proofs where one for $\mathbb{F}_2$ and the other for $\mathbb{F}_p$, we can construct authenticated values $([r_0]_2, \dots, [r_{m-1}]_2)$ and $[r]_p$ with communication of $O(\log p)$ bits in total. Nevertheless, if the prover is malicious, then there may be an inconsistency (i.e., $r \neq \sum_{h=0}^{m-1} r_h \cdot 2^h \bmod p$). Now, our task is to check the consistency of N faulty zk-edaBits computed as above, where N is the number of zk-edaBits needed. This could be done using the “cut-and-bucketing” technique similar to prior work [5, 32, 52]. Specifically, we let a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ generate extra $N(B-1) + c$ faulty zk-edaBits, where B, c are two parameters. Then two parties use a random permutation to shuffle $NB + c$ faulty zk-edaBits. The last c faulty zk-edaBits are opened, and their correctness is checked by $\mathcal{V}$. Next, the remaining NB faulty

zk-edaBits are partitioned into N buckets with each of size B . Finally, for each bucket, the parties perform the “combine-and-open” check $B - 1$ times of between the first zk-edaBit in the bucket and the other $B - 1$ zk-edaBits. See Section 4.1 for more details and optimization.

Arithmetic-Boolean conversions using zk-edaBits. In Figure 3, we define the functionality for converting authenticated wire values between arithmetic and Boolean circuits. We will use random zk-edaBits to realize the conversion of authenticated values between arithmetic and Boolean wires.

Given a random zk-edaBit $([r_0]_2, \dots, [r_{m-1}]_2, [r]_p)$ and an authenticated value $[x]_p$ to be converted, $\mathcal{P}$ and $\mathcal{V}$ can open a *masked* value $[z]_p = [x]_p - [r]_p$, and call functionality $\mathcal{F}_{\text{authZK}}$ with only circuit-based commands to compute a modular-addition circuit on a public input $(z_0, \dots, z_{m-1})$ and a secret input $([r_0]_2, \dots, [r_{m-1}]_2)$ so that they obtain $([x_0]_2, \dots, [x_{m-1}]_2)$, where $(z_0, \dots, z_{m-1})$ is the bit-decomposition of z . From $z = x - r \pmod p$ and $\sum_{h=0}^{m-1} z_h \cdot 2^h = z \pmod p$, one can easily verify that $(x_0, \dots, x_{m-1})$ is the bit-decomposition of x . The other direction can be constructed in a similar way.

In general, we need to convert multiple authenticated values between arithmetic and Boolean wires, and thus can open multiple authenticated values *in a batch* to reduce the communication cost. In Section 4.2, we will provide the full description of our protocols and formal proofs of security.

3.2 Conversion from Publicly Committed Values to Privately Authenticated Values

Our second task is to convert from non-interactive commitments (publicly available to all parties) to authenticated values (privately available between two parties). The former is suitable for committing values in a public repository, while the latter is better for efficient sVOLE-based ZK proofs and also compatible with the above conversion between arithmetic and Boolean circuits.

Our conversion uses the commitment scheme in a non-black-box way: a prover $\mathcal{P}$ first authenticates to a verifier $\mathcal{V}$ the committed values as well as the decommitment, and then proves in zero-knowledge that the authenticated values satisfy the opening of the public commitment. This establishes a connection between a public commitment and privately authenticated values. The efficiency of the protocol crucially relies on the size of the circuit to represent the opening of a commitment scheme. If we use cryptographic hash functions like SHA-256, it would require more than 22,000 AND gates to commit a 512-bit message, averaging to 42 gates per bit. One can also use LowMC [2] as a block cipher with the 256-bit key and 256-bit block length, where LowMC allows much less AND gates than standard block ciphers such as AES. When being modeled as an ideal cipher, LowMC can be converted to a suitable hash function using the Merkle–Damgård structure. However, the computation complexity in this case

could be very high as we need to calculate a lot of matrix multiplications with random bits.

To minimize the circuit size, we use a “hybrid commitment” scheme: to commit a set of messages $\{\mathbf{x}_i\}_{i \in [1, \ell]}$ with $\mathbf{x}_i \in \{0, 1\}^m$, we first pick a random key $sk \leftarrow \{0, 1\}^\lambda$, commit to this key by using a slow commitment scheme (e.g., $H(sk, r)$ for a random oracle H and a randomness r), and then commit the messages as $\mathbf{c}_i := \text{PRF}(sk, i) \oplus \mathbf{x}_i$ for $i \in [1, \ell]$, where PRF is a standard pseudorandom function. The security of this hybrid commitment scheme can be reduced to the security of the slow commitment scheme as well as the pseudo-randomness of PRF. What’s more, if the slow commitment scheme is extractable, then the overall commitment scheme is also extractable. Note that we cannot equivocate $\mathbf{x}_i$ to any vector if we use the natural “open” algorithm with sending sk and $\mathbf{x}_i$, as the function PRF is fixed. However, we can make this commitment scheme equivocal in an interactive way: we prove knowledge of sk and $\mathbf{x}_i$ such that all relationships hold, and convert the committed value to an authenticated value in zero knowledge. Besides, we easily extend the above hybrid commitment scheme to support committed values over any field $\mathbb{F}_q$ by extending the output range of PRF to $\mathbb{F}_q^m$.

To reduce the number of AND gates for PRF, we choose to use LowMC to instantiate PRF. To obtain faster computation, we adopt a smaller block size (i.e., 64 bits). As a result, our protocol can convert 18,000 64-bit committed values (144 KB in total) to authenticated values in a second.

3.3 Optimizations for ML Applications

To make ZK proofs of ML applications practical, we also propose several optimizations specifically to reduce the overhead of some key ML components and to integrate with TensorFlow [1]. Detailed descriptions can be found in Section 6.

Matrix multiplication. Directly proving matrix multiplication in zero knowledge would require $O(n^3)$ number of multiplications, which could be improved to $O(n^{2.8})$ (or even lower) based on a better algorithm [48]. Although the prover time has to be linear to this complexity, we could reduce the circuit size for ZK proofs significantly. Suppose in a certain stage of the ZK protocol, a prover $\mathcal{P}$ wants to prove the relation that $\mathbf{A} \cdot \mathbf{B} = \mathbf{C}$ with $\mathbf{A} \in \mathbb{F}_q^{n \times m}$, $\mathbf{B} \in \mathbb{F}_q^{m \times \ell}$, $\mathbf{C} \in \mathbb{F}_q^{n \times \ell}$, where $\mathbf{A}, \mathbf{B}, \mathbf{C}$ have been committed using authenticated values resulting in $[\mathbf{A}], [\mathbf{B}], [\mathbf{C}]$. By generalizing the Freivalds algorithm [31], we use a random-linear-combination approach to prove that $\mathbf{A} \cdot \mathbf{B} = \mathbf{C}$ holds. Specifically, we can let a verifier $\mathcal{V}$ sample two uniformly random vectors $\mathbf{u} \in (\mathbb{F}_{q^k})^n$, $\mathbf{v} \in (\mathbb{F}_{q^k})^\ell$. Instead of directly proving $[\mathbf{A}] \cdot [\mathbf{B}] = [\mathbf{C}]$, we can prove:

$$\mathbf{u}^\top \cdot [\mathbf{A}] \cdot [\mathbf{B}] \cdot \mathbf{v} = \mathbf{u}^\top \cdot [\mathbf{C}] \cdot \mathbf{v}.$$

Now, the parties can locally compute vectors of authenticated values $[\mathbf{x}]^\top = [\mathbf{u}^\top \cdot \mathbf{A}] \in (\mathbb{F}_{q^k})^m$, $[\mathbf{y}] = [\mathbf{B} \cdot \mathbf{v}] \in (\mathbb{F}_{q^k})^m$ and $[\mathbf{z}] = [\mathbf{u}^\top \cdot \mathbf{C} \cdot \mathbf{v}] \in \mathbb{F}_{q^k}$. Thus, they only need to prove in

zero-knowledge that $[\mathbf{x}]^\top \cdot [\mathbf{y}] = [z]$, which takes only the communication of $O(k \log q)$ bits using the latest ZK proof [54], where k is an integer satisfying $q^k > 2^p$. Note that this ZK protocol [54] allows to prove the statements over $\mathbb{F}_{q^k}$.

Fixed-point and floating-point conversions. The above matrix-multiplication protocol only works over a field. However, in the neural-network inference, all operations are for real numbers. To address this discrepancy, we use both fixed-point and floating-point encodings of real numbers at different stages of our protocol. Firstly, we encode a signed, fixed-point number x with $-\frac{p-1}{2^{s+1}} \leq x \leq \frac{p-1}{2^{s+1}}$ into a field element in $[-\frac{p-1}{2}, \frac{p-1}{2}]$ by computing $[2^s \cdot x]$, where $p > 2$ is a prime and $s \in \mathbb{N}$ is a precision parameter. Then we easily encode field elements in $[-\frac{p-1}{2}, \frac{p-1}{2}]$ into field elements in $[0, p-1]$. In this way, the addition and multiplication of fixed-point numbers are the same as addition and multiplication over field $\mathbb{F}_p$, as long as there is no overflow. One caveat is that overflow can happen quickly if the multiplication depth is high. Fortunately, for matrix multiplication, the multiplication depth is 1. After linear layers, we usually need to perform many non-linear operations like Batch Normalization (which needs square root and inverse), SoftMax (which additionally needs exponentiation), ReLU (which additionally needs comparison), etc. To support these operations efficiently and accurately, we convert between fixed-point numbers and IEEE-754 compliant floating-point numbers using functionality $\mathcal{F}_{\text{authZK}}$ with only circuit-based commands, such that non-linear operations can be performed in zero-knowledge.

Integration with TensorFlow. To easily implement complicated neural networks, we integrated our backend protocol with TensorFlow [1], so that existing TensorFlow neural network implementations can be directly executed in our protocol, while keeping the TensorFlow interfaces unchanged. In particular, we implemented a set of common operators that are needed and hook them with TensorFlow using a dynamic pass. Due to our use of floating-point numbers in the non-linear layers, adding more operators is fairly straightforward. See Section 6.3 for more details.

4 Arithmetic-Boolean Conversion for Zero-Knowledge Proofs

In this section, we provide full details on how to construct *ZK-friendly extended doubly authenticated bits* (zk-edaBits) efficiently, and then show how to use them to securely realize conversions between arithmetic and Boolean circuits.

4.1 Extended Doubly Authenticated Bits for Zero-Knowledge Proofs

As described in Section 3.1, zk-edaBit is a key tool in this work to efficiently perform conversions between arithmetic

Functionality $\mathcal{F}_{\text{zk-edaBits}}$

This functionality is parameterized by a prime $p > 2$ and an integer $k \geq 1$ with $p^k \geq 2^p$. Let $m = \lceil \log p \rceil$.

Initialize: On input (init) from $\mathcal{P}$ and $\mathcal{V}$, sample $\Delta \leftarrow \mathbb{F}_{2^\lambda}$ and $\Gamma \leftarrow \mathbb{F}_{p^k}$ if $\mathcal{V}$ is honest, and receive $\Delta \in \mathbb{F}_{2^\lambda}$ and $\Gamma \in \mathbb{F}_{p^k}$ from the adversary otherwise. Store two global keys (Δ, Γ) and send them to $\mathcal{V}$, and ignore all subsequent (init) commands.

Generate random ZK-friendly edaBits: On input (random, id, id₀, ..., id_{m-1}) from two parties $\mathcal{P}$ and $\mathcal{V}$ where id, id₀, ..., id_{m-1} are fresh identifiers, generate a random zk-edaBit $([r_0]_2, \dots, [r_{m-1}]_2, [r]_p)$ with $r_i \in \mathbb{F}_2$ for $i \in [0, m)$ and $r = \sum_{i=0}^{m-1} r_i \cdot 2^i \in \mathbb{F}_p$ as follows:

1. If $\mathcal{P}$ is honest, sample $r \leftarrow \mathbb{F}_p$. Otherwise, receive $r \in \mathbb{F}_p$ from the adversary. Decompose r to $(r_0, \dots, r_{m-1})$ such that $r = \sum_{i=0}^{m-1} r_i \cdot 2^i \pmod p$.
2. Execute $[r_i]_2 \leftarrow \text{Auth}(r_i, 2)$ for $i \in [0, m)$ and $[r]_p \leftarrow \text{Auth}(r, p)$, where the macro $\text{Auth}(\cdot)$ is described in Figure 4. Thus, the two parties obtain $([r_0]_2, \dots, [r_{m-1}]_2, [r]_p)$.
3. Store (id, p , $[r]_p$) and (id_i, 2, $[r_i]_2$) for $i \in [0, m)$.

Figure 5: **Functionality for ZK-friendly extended doubly authenticated bits.**

and Boolean circuits. A zk-edaBit consists of a set of m authenticated bits $([r_0]_2, \dots, [r_{m-1}]_2)$ along with a *random* authenticated value $[r]_p$ such that $r = \sum_{h=0}^{m-1} r_h \cdot 2^h \in \mathbb{F}_p$. We provide the ideal functionality for zk-edaBits in Figure 5.

A prover $\mathcal{P}$ and a verifier $\mathcal{V}$ can generate faulty zk-edaBits by calling functionality $\mathcal{F}_{\text{authZK}}$, and then use a “cut-and-bucketing” technique to check the consistency of resulting zk-edaBits. Recall that the overview of our technique has been described in Section 3.1. Thus, we directly provide the details of our zk-edaBits protocol in Figure 6. In this protocol, the prover and verifier use $\mathcal{F}_{\text{authZK}}$ with only circuit-based commands to compute a Boolean circuit AdderModp , which efficiently realizes the module-addition computation that adds two m -bit integers and then modules a prime p .

Theorem 1. *Protocol $\Pi_{\text{zk-edaBits}}$ shown in Figure 6 UC-realizes functionality $\mathcal{F}_{\text{zk-edaBits}}$ in the presence of a static, malicious adversary with statistical error at most $\left(\frac{N(B-1)+c}{B-1} \right)^{-1} + \frac{1}{p^k}$ in the $\mathcal{F}_{\text{authZK}}$ -hybrid model.*

The proof of this theorem can be found in the full version [53]. Given the number N of zk-edaBits, we can choose suitable parameters B and c such that $\left(\frac{N(B-1)+c}{B-1} \right)^{-1} \leq$

Protocol $\Pi_{\text{zk-edaBits}}$

Parameters: Let $p > 2$ be a prime, $m = \lceil \log p \rceil$ and $k \in \mathbb{N}$ with $p^k \geq 2^p$. Two parties want to generate N zk-edaBits. Let B, c be some parameters to be specified later and $\ell = NB + c$.

Initialize: $\mathcal{P}$ and $\mathcal{V}$ send (init) to $\mathcal{F}_{\text{authZK}}$, which returns two uniform global keys to $\mathcal{V}$.

Generating random zk-edaBits:

1. The parties generate random authenticated values $[r^i]_p$ for $i \in [1, \ell]$. Then, for $i \in [1, \ell]$, $\mathcal{P}$ decomposes r^i to $(r_0^i, \dots, r_{m-1}^i)$ such that $r^i = \sum_{h=0}^{m-1} r_h^i \cdot 2^h \pmod p$.
2. For $i \in [1, \ell]$, $\mathcal{P}$ inputs $(r_0^i, \dots, r_{m-1}^i) \in \mathbb{F}_2^m$ to $\mathcal{F}_{\text{authZK}}$, which returns $([r_0^i]_2, \dots, [r_{m-1}^i]_2)$ to the parties.
3. Place the first N zk-edaBits into N buckets in order, where each bucket has exactly one zk-edaBit. Then, $\mathcal{V}$ samples a random permutation π and sends it to $\mathcal{P}$. Use π to permute the remaining $\ell - N$ zk-edaBits.
4. The parties check that the last c zk-edaBits are correctly computed and abort if not. Divide the remaining $N(B - 1)$ (unopened) zk-edaBits into N buckets accordingly, such that each bucket has B zk-edaBits.
5. For each bucket, both parties choose the first zk-edaBit $([r_0]_2, \dots, [r_{m-1}]_2, [r]_p)$ (that is placed into the bucket in the step 3), and for every other zk-edaBit $([s_0]_2, \dots, [s_{m-1}]_2, [s]_p)$ in the same bucket, execute the following check:
 - (a) Compute $[t]_p := [r]_p + [s]_p$, and then execute $([t_0]_2, \dots, [t_{m-1}]_2) := \text{AdderModp}([r_0]_2, \dots, [r_{m-1}]_2, [s_0]_2, \dots, [s_{m-1}]_2)$ by calling functionality $\mathcal{F}_{\text{authZK}}$, where AdderModp is the modular-addition circuit, and $\sum_{h=0}^{m-1} t_h \cdot 2^h = \sum_{h=0}^{m-1} r_h \cdot 2^h + \sum_{h=0}^{m-1} s_h \cdot 2^h \pmod p$.
 - (b) Execute the BatchCheck procedure on $([t_0]_2, \dots, [t_{m-1}]_2)$ to obtain $(t_0, \dots, t_{m-1})$, and then compute $t' := \sum_{h=0}^{m-1} t_h \cdot 2^h \pmod p$.
 - (c) Execute the CheckZero procedure on $[t]_p - t'$ to verify that $t = t'$.
6. If any check fails, $\mathcal{V}$ aborts. Otherwise, the parties output the first zk-edaBit from each of the N buckets.

Figure 6: **Protocol for generating ZK-friendly edaBits in the $\mathcal{F}_{\text{authZK}}$ -hybrid model.**

2^{-p} . For example, when $N = 10^6$, we can choose $B = 3$ and $c = 2$, and achieve at least 40-bit statistical security.

Protocol $\Pi_{\text{Convert}}^{\text{A2B}}$

Let $p > 2$ be a prime and $m = \lceil \log p \rceil$.

Initialize: $\mathcal{P}$ and $\mathcal{V}$ send (init) to $\mathcal{F}_{\text{zk-edaBits}}$, which returns two uniform global keys to $\mathcal{V}$.

Input: The parties have an authenticated value $[x]_p$.

Convert: $\mathcal{P}$ and $\mathcal{V}$ convert an authenticated value over field $\mathbb{F}_p$ into m authenticated bits as follows:

1. Call functionality $\mathcal{F}_{\text{zk-edaBits}}$, which returns $([r_0]_2, \dots, [r_{m-1}]_2, [r]_p)$ to the parties.
2. Compute $[z]_p := [x]_p - [r]_p$, and then execute the BatchCheck procedure on $[z]_p$ to obtain z .
3. Decompose z as $(z_0, \dots, z_{m-1})$ such that $z = \sum_{h=0}^{m-1} z_h \cdot 2^h \pmod p$, and then compute $([x_0]_2, \dots, [x_{m-1}]_2) := \text{AdderModp}(z_0, \dots, z_{m-1}, [r_0]_2, \dots, [r_{m-1}]_2)$ by calling $\mathcal{F}_{\text{authZK}}$ where $z_0, \dots, z_{m-1}$ are public constants.
4. Output $([x_0]_2, \dots, [x_{m-1}]_2)$.

Figure 7: **Protocol for converting from arithmetic to Boolean circuits in the $(\mathcal{F}_{\text{zk-edaBits}}, \mathcal{F}_{\text{authZK}})$ -hybrid model.**

4.2 Arithmetic-Boolean Conversion Protocols

Using functionality $\mathcal{F}_{\text{zk-edaBits}}$ efficiently realized in the previous sub-section, we propose two efficient protocols to convert authenticated wire values from an arithmetic circuit to a Boolean circuit and to convert in another direction. In the two protocols, the prover and verifier would also use functionality $\mathcal{F}_{\text{authZK}}$ with only circuit-based commands to compute a Boolean circuit AdderModp . In both protocols, we assume that $\mathcal{F}_{\text{zk-edaBits}}$ shares the same initialization procedure with $\mathcal{F}_{\text{authZK}}$, and thus the same global keys are used in the two functionalities. This is the case, when we use the protocol $\Pi_{\text{zk-edaBits}}$ shown in Figure 6 to UC-realize $\mathcal{F}_{\text{zk-edaBits}}$ in the $\mathcal{F}_{\text{authZK}}$ -hybrid model. We also assume that $\mathcal{F}_{\text{authZK}}$ can use authenticated values generated by $\mathcal{F}_{\text{zk-edaBits}}$. This is easy to be realized by viewing $\mathcal{F}_{\text{zk-edaBits}}$ as a part of $\mathcal{F}_{\text{authZK}}$.

We provide the full details about the conversion from arithmetic to Boolean circuits in Figure 7. In Figure 8, we describe in details how to perform an efficient conversion from Boolean to arithmetic circuits.

Below, we prove the security of the two protocols in the following theorems.

Theorem 2. *Protocol $\Pi_{\text{Convert}}^{\text{A2B}}$ UC-realizes the convertA2B command of functionality $\mathcal{F}_{\text{authZK}}$ in the presence of a static, malicious adversary with statistical error $1/p^k$ in the $(\mathcal{F}_{\text{zk-edaBits}}, \mathcal{F}_{\text{authZK}})$ -hybrid model.*

Theorem 3. *Protocol $\Pi_{\text{Convert}}^{\text{B2A}}$ UC-realizes the convertB2A*

Protocol $\Pi_{\text{Convert}}^{\text{B2A}}$

Let $p > 2$ be a prime and $m = \lceil \log p \rceil$.

Initialize: $\mathcal{P}$ and $\mathcal{V}$ send (init) to $\mathcal{F}_{\text{zk-edaBits}}$, which returns two uniform global keys to $\mathcal{V}$.

Input: Two parties $\mathcal{P}$ and $\mathcal{V}$ hold m authenticated bits $[x_0]_2, \dots, [x_{m-1}]_2$.

Convert: $\mathcal{P}$ and $\mathcal{V}$ convert m authenticated bits into one authenticated value over field $\mathbb{F}_p$ as follows:

1. Call functionality $\mathcal{F}_{\text{zk-edaBits}}$, which returns $([r_0]_2, \dots, [r_{m-1}]_2, [r]_p)$ to the parties.
2. Compute $([z_0]_2, \dots, [z_{m-1}]_2) := \text{AdderModp}([x_0]_2, \dots, [x_{m-1}]_2, [r_0]_2, \dots, [r_{m-1}]_2)$ by calling functionality $\mathcal{F}_{\text{authZK}}$, such that $\sum_{h=0}^{m-1} z_h \cdot 2^h = \sum_{h=0}^{m-1} x_h \cdot 2^h + \sum_{h=0}^{m-1} r_h \cdot 2^h \pmod p$.
3. Execute the BatchCheck procedure on $([z_0]_2, \dots, [z_{m-1}]_2)$ to obtain $(z_0, \dots, z_{m-1})$, and then compute $z := \sum_{h=0}^{m-1} z_h \cdot 2^h \pmod p$.
4. Compute and output $[x]_p := z - [r]_p$.

Figure 8: **Protocol for converting from Boolean to arithmetic circuits in the $(\mathcal{F}_{\text{zk-edaBits}}, \mathcal{F}_{\text{authZK}})$ -hybrid model.**

command of $\mathcal{F}_{\text{authZK}}$ in the presence of a static, malicious adversary in the $(\mathcal{F}_{\text{zk-edaBits}}, \mathcal{F}_{\text{authZK}})$ -hybrid model.

The proofs of the above two theorems can be found in the full version of this paper [53].

Optimization using circuit-based zk-edaBits. In the conversion protocols described as above, a prover $\mathcal{P}$ and a verifier $\mathcal{V}$ generate random zk-edaBits using functionality $\mathcal{F}_{\text{zk-edaBits}}$ in the preprocessing phase, and then convert authenticated values between arithmetic and Boolean circuits using these random zk-edaBits in the online phase.

We can use an alternative approach to convert authenticated values between arithmetic and Boolean circuits, and obtain better whole efficiency but larger online cost. Specifically, for authenticated bits $[x_0]_2, \dots, [x_{m-1}]_2$ on m output wires of a Boolean circuit, $\mathcal{P}$ can compute $x := \sum_{h=0}^{m-1} x_h \cdot 2^h \pmod p$ locally. Then, $\mathcal{P}$ sends (input, x, p) to $\mathcal{F}_{\text{authZK}}$ and $\mathcal{V}$ sends (input, p) to $\mathcal{F}_{\text{authZK}}$, which returns $[x]_p$ to the parties. Similarly, the parties can also convert an authenticated value $[x]_p$ on an output wire of an arithmetic circuit into m authenticated bits $[x_0]_2, \dots, [x_{m-1}]_2$, by calling the (input) command of $\mathcal{F}_{\text{authZK}}$. In this way, two parties can create N circuit-based zk-edaBits for some integer N . However, in the circuit-based zk-edaBits, a malicious prover may cause the field elements over $\mathbb{F}_p$ are inconsistent with corresponding bits. Verifier $\mathcal{V}$ can check the consistency of these circuit-based zk-edaBits

using the cut-and-bucketing technique. Specifically, in the online phase, two parties can execute the checking procedure shown in Figure 6 to check the consistency of these circuit-based zk-edaBits by sacrificing $(B-1)N + c$ random zk-edaBits generated in the preprocessing phase. Using this optimization, for computing N circuit-based zk-edaBits, we can save N random zk-edaBits and N evaluations of circuit AdderModp in terms of the whole efficiency, but increase the online cost by a factor of $B-1$.

5 Converting Publicly Committed Values to Privately Authenticated Values

The second type of conversions that we would like to study is the conversion from publicly committed data to privately authenticated data. Here, publicly committed data referred to those committed with a short digest, which can be published on something that can be modeled as a bulletin board (e.g., well-established websites or some blockchain). Privately authenticated data refers to the values only known by a prover that are authenticated by a designated verifier based on IT-MACs, and thus can be efficiently used to prove any mixed arithmetic-Boolean circuit using the recent ZK protocols [7, 27, 52, 54] and our arithmetic-Boolean conversion protocols. The conversion from publicly committed data to privately authenticated data will allow us to efficiently prove statements on consistent committed data to multiple different verifiers for multiple times.

Our commitment-authentication conversion protocol. We present our efficient conversion protocol in Figure 9. This protocol consists of two phases: 1) generating a non-interactive commitment and 2) converting publicly committed values to privately authenticated values in an interactive manner. To commit a large volume of data or different types of data, we divide them into pieces, where the i -th piece is denoted by $\mathbf{x}_i \in \mathbb{F}_q^m$ with a prime $q \geq 2$ and a parameter m . Then, we let the prover sample a random key sk and a uniform randomness r both in $\{0, 1\}^\lambda$. Our commitment consists of $\text{com}_0 = H(sk, r)$ and $\mathbf{c}_i = \text{PRF}(sk, i) + \mathbf{x}_i \in \mathbb{F}_q^m$ for all $i \in [1, \ell]$ with some $\ell \in \mathbb{N}$, where H is a random oracle and PRF is a pseudorandom function. To perform conversion, the prover $\mathcal{P}$ proves knowledge of sk and $\mathbf{x}_i$, such that com_0 and $\mathbf{c}_i$ are computed with the key and data piece. Since $\mathbf{c}_i$ can be put in the public domain, one can further reduce the size of the overall commitment by computing a Merkle tree on top of all $\mathbf{c}_i$'s. In this way, the commitment only has a size of 4λ bits, including com_0 and the root of the Merkle tree (i.e., com_1).

Since key $sk \in \{0, 1\}^\lambda$ has a high entropy, we can actually remove the randomness r . That is, the prover can just set $H(sk)$ as com_0 in the commitment phase and prove $\text{com}_0 = H([sk]_2)$ in the conversion phase. This will slightly improve the efficiency of this protocol.

Protocol $\Pi_{\text{NCom} \rightarrow [\cdot]}$

Let $q \geq 2$ be a prime. Let $H : \{0, 1\}^* \rightarrow \{0, 1\}^{2\lambda}$ be a hash function modeled as a random oracle, and $\text{PRF} : \{0, 1\}^\lambda \times \{0, 1\}^\lambda \rightarrow \mathbb{F}_q^m$ be a pseudorandom function.

Compute a public commitment: A prover $\mathcal{P}$ computes and publishes a *non-interactive* commitment on values:

1. Sample $sk, r \leftarrow \{0, 1\}^\lambda$; compute $\text{com}_0 := H(sk, r)$.
2. On input $\mathbf{x}_1, \dots, \mathbf{x}_\ell \in \mathbb{F}_q^m$ with $\ell, m \in \mathbb{N}$, compute $\mathbf{c}_i := \text{PRF}(sk, i) + \mathbf{x}_i \in \mathbb{F}_q^m$ for $i \in [1, \ell]$.
3. Compute $d_i := H(\mathbf{c}_i)$ for all $i \in [1, \ell]$; build a Merkle tree on these values using H with com_1 as the root.
4. Publish the commitment $(\text{com}_0, \text{com}_1)$.

Initialize: Prover $\mathcal{P}$ and a verifier $\mathcal{V}$ send (init) to $\mathcal{F}_{\text{authZK}}$, which returns two uniform global keys to $\mathcal{V}$.

Convert committed values into authenticated values:

This procedure can be executed multiple times. For some $i \in [1, \ell]$, $\mathcal{P}$ and $\mathcal{V}$ convert a committed value $\mathbf{x}_i \in \mathbb{F}_q^m$ only known by $\mathcal{P}$ to m authenticated values $[x_{i,1}], \dots, [x_{i,m}]$:

1. Let path_i be the set containing all siblings of the nodes in the path from the i -th leaf to the root com_1 . Prover $\mathcal{P}$ sends $(\mathbf{c}_i, \text{path}_i)$ to $\mathcal{V}$, who verifies that $H(\mathbf{c}_i)$ is a leaf node rooted in com_1 .
2. By calling functionality $\mathcal{F}_{\text{authZK}}$, the parties obtain authenticated bit-vectors $[\mathbf{sk}]_2, [\mathbf{r}]_2$ on key sk and randomness r , and then $\mathcal{P}$ proves in zero-knowledge that $\text{com}_0 = H([\mathbf{sk}]_2, [\mathbf{r}]_2)$.
3. The parties call functionality $\mathcal{F}_{\text{authZK}}$ to compute $([x_{i,1}], \dots, [x_{i,m}]) \leftarrow \mathbf{c}_i - \text{PRF}([\mathbf{sk}]_2, i) \in \mathbb{F}_q^m$, and then output $\{[x_{i,j}]\}_{j \in [1, m]}$.

Figure 9: **Protocol for converting committed values into authenticated values in the $\mathcal{F}_{\text{authZK}}$ -hybrid model.**

Theorem 4. Let H be a random oracle and PRF be a pseudorandom function. Then protocol $\Pi_{\text{NCom} \rightarrow [\cdot]}$ shown in Figure 9 UC-realizes the `convertC2A` command of functionality $\mathcal{F}_{\text{authZK}}$ in the presence of a static, malicious adversary in the $\mathcal{F}_{\text{authZK}}$ -hybrid model.

Below, we discuss the intuition of the above theorem and leave the full formal proof to the full version [53]. We commit to sk using a standard UC commitment in the random-oracle model, and so sk is computationally hiding, meaning that $\text{PRF}(sk, i)$ for all $i \in [1, \ell]$ are indistinguishable from uni-

formly random values in $\mathbb{F}_q^m$. In the $\mathcal{F}_{\text{authZK}}$ -hybrid model, the ZK proof does not reveal any information of committed values. Overall, the committed data is hidden. In the proof of security, the simulator can extract the key sk from com_0 in the random-oracle model. Once sk was extracted, the simulator can easily recover $\mathbf{x}_i$ by decrypting $\mathbf{c}_i$ for $i \in [1, \ell]$. This also implies the binding property. Together with the soundness of the ZK protocol realizing $\mathcal{F}_{\text{authZK}}$, we can ensure the consistency between authenticated values and committed values.

Note that this commitment $(\text{com}_0, \text{com}_1)$ itself is not equivocal if we use the natural “open” algorithm that sends (sk, r) : although it is possible to equivocate the key sk to any value by programming the random oracle, the function PRF is fixed. Equivocating from $\mathbf{x}_i$ to $\mathbf{x}'_i$ would require finding a key sk' such that $\text{PRF}(sk', i) - \text{PRF}(sk, i) = \mathbf{x}_i - \mathbf{x}'_i$ over $\mathbb{F}_q^m$. However, we can make it equivocal by an interactive opening: instead of directly sending (sk, r) , we can send $\mathbf{c}_i$ and the corresponding path that can be verified with com_1 , and prove knowledge of a key sk and a randomness r such that $\text{com}_0 = H(sk, r)$ and the other relationship on $\mathbf{c}_i$ hold. In this way, we can use the zero-knowledge property to equivocate the commitment.

Instantiation of PRF. We use LowMC [2] to instantiate PRF for reducing circuit complexity. One issue with LowMC is that it contains a lot of XOR gates. Although they are free cryptographically, the computation complexity can be fairly high. We adopt the following optimizations for competitive performance:

- Similar to the signature scheme Picnic [56], we need to run PRF on a single key for many times, and thus can precompute the matrix multiplication about the key *only once* and use it for all PRF evaluations.
- We pick the block size as 64 bits to further reduce the number of XOR operations. The resulting protocol is highly efficient, and can convert 18,000 publicly committed data blocks (totally 144KB) to authenticated values per second.
- To reduce the number of rounds in LowMC, we choose the data complexity to be 2^{30} blocks, which is sufficient to commit 8 GB data. If the data is larger than that, we can just pick a new PRF key and commit this key.

Comparing with other candidates. We briefly discuss the concrete efficiency of our protocol for one commitment-authentication conversion, and compares it with other alternatives shown in Table 1. Here, we ignore the efficiency comparison for the commitment-generation phase, as it needs to be executed only once.

For SHA-256 and LowMC-256, they refer to building a hash function modeled as a random oracle, and further construct a commitment on message x via $H(x, r)$ with a randomness r . For SHA-256, one invocation takes 22573 AND gates and can commit 256 bits of messages. For LowMC-256, we first pick a LowMC block cipher with 256-bit key and block

Scheme	This work	SHA-256	LowMC-256
Time (μs)	55	395	≥ 1000
Comm. (bits)	62	705	49

Table 1: **Efficiency comparison between our protocol and alternative protocol with natural commitments.** Running time in microsecond (μs) is based on two Amazon EC2 machines of type m5.2xlarge.

sizes, and then use Davies–Meyer to build a hash function. The SHA-256 method requires a lot of communication due to a large circuit size. The LowMC-256 approach is significantly slower compared to ours because: 1) our 64-bit block cipher only computes 64-bit matrix multiplication, but LowMC-256 needs 256-bit matrix multiplication meaning 16 times more operations; 2) we only need 11 rounds but LowMC-256 needs 53 rounds; 3) we can use a fixed key for all messages but LowMC-256 needs to rekey for every block of the message.

Conversion from authenticated values to publicly committed values. In some applications, two parties $\mathcal{P}$ and $\mathcal{V}$ may want to convert authenticated values (say, output by some MPC protocol) into a public commitment on the same values. Based on the protocol $\Pi_{\text{NCom} \rightarrow [\cdot]}$ shown in Figure 9, this is easy to be realized by the following execution:

1. To convert authenticated values $\{[x_{i,j}]\}_{i \in [1,\ell], j \in [1,m]}$ into publicly committed values, $\mathcal{P}$ commits these vectors $(x_{i,1}, \dots, x_{i,m})$ for $i \in [1,\ell]$ by executing the commitment-generation phase of protocol $\Pi_{\text{NCom} \rightarrow [\cdot]}$. Then $\mathcal{P}$ publishes the resulting commitment $(\text{com}_0, \text{com}_1)$.
2. Then, $\mathcal{P}$ and $\mathcal{V}$ execute protocol $\Pi_{\text{NCom} \rightarrow [\cdot]}$ to convert commitment $(\text{com}_0, \text{com}_1)$ into authenticated values $\{[x'_{i,j}]\}_{i \in [1,\ell], j \in [1,m]}$.
3. The parties call the CheckZero procedure on $[x'_{i,j}] - [x_{i,j}]$ for all $i \in [1,\ell], j \in [1,m]$, and abort if the check fails.

6 More Optimizations for ML Applications

In this section, we will discuss several optimizations for key components in the *machine learning* (ML) applications and how they are connected. Then, we describe how to support various types of ML algorithms by extending TensorFlow [1].

6.1 Optimizing Matrix Multiplication

By generalizing the Freivalds algorithm [31], we propose a ZK protocol to prove matrix multiplication with dimension $n \times n$ over a field $\mathbb{F}_q$ (for any prime $q \geq 2$), which only needs to prove n private multiplications rather than n^3 using a naive algorithm. Since the intuition of the protocol has been discussed in Section 3.3, we directly present the ZK protocol in the $\mathcal{F}_{\text{authZK}}$ -hybrid model in Figure 10.

Protocol Π_{MatMul}

Inputs: A prover $\mathcal{P}$ and a verifier $\mathcal{V}$ have three authenticated matrices $[\mathbf{A}], [\mathbf{B}]$ and $[\mathbf{C}]$, where $\mathbf{A} \in \mathbb{F}_q^{n \times m}, \mathbf{B} \in \mathbb{F}_q^{m \times \ell}$ and $\mathbf{C} \in \mathbb{F}_q^{n \times \ell}$.

Protocol execution: $\mathcal{P}$ proves in zero-knowledge that $\mathbf{A} \cdot \mathbf{B} = \mathbf{C}$ holds by interacting with $\mathcal{V}$ as follows:

1. $\mathcal{V}$ samples $\mathbf{u} \leftarrow (\mathbb{F}_{q^k})^n, \mathbf{v} \leftarrow (\mathbb{F}_{q^k})^\ell$, and then sends them to $\mathcal{P}$.
2. $\mathcal{P}$ and $\mathcal{V}$ compute $[\mathbf{x}]^\top := \mathbf{u}^\top \cdot [\mathbf{A}]$ and $[\mathbf{y}] := [\mathbf{B}] \cdot \mathbf{v}$ locally (this can be also done by calling the (lincomb) command of $\mathcal{F}_{\text{authZK}}$). Both parties also compute $[z] := \mathbf{u}^\top \cdot [\mathbf{C}] \cdot \mathbf{v}$.
3. The parties compute $[z'] := [\mathbf{x}]^\top \cdot [\mathbf{y}]$ by calling functionality $\mathcal{F}_{\text{authZK}}$, where $z' = \mathbf{x}^\top \cdot \mathbf{y}$.
4. Both parties execute the CheckZero procedure on $[z] - [z']$ to verify that $z = z'$. If the check fails, $\mathcal{V}$ outputs false and aborts; otherwise, it outputs true.

Figure 10: **Zero-knowledge protocol for proving matrix multiplication in the $\mathcal{F}_{\text{authZK}}$ -hybrid model.** Before running this protocol, $\mathcal{P}$ and $\mathcal{V}$ have computed the authenticated values on all entries in the matrices to be proven by calling the (input) command of $\mathcal{F}_{\text{authZK}}$.

In the following theorem, we prove the security of this protocol, where we refer the reader to [52] for the standard ZK functionality. The detailed proof of this theorem can be found in the full version [53].

Theorem 5. *Protocol Π_{MatMul} shown in Figure 10 UC-realizes the standard ZK functionality $\mathcal{F}_{\text{ZK}}$ in the presence of a static, malicious adversary with soundness error $3/q^k$ in the $\mathcal{F}_{\text{authZK}}$ -hybrid model.*

Further optimizations. We can further optimize the protocol shown in Figure 10 by letting the verifier send a random seed to the prover and then the two parties compute $\mathbf{u}$ and $\mathbf{v}$ by applying a random oracle to the seed.

In protocol Π_{MatMul} , the parties compute $[z'] = [\mathbf{x}]^\top \cdot [\mathbf{y}]$ by calling the (mult) command of $\mathcal{F}_{\text{authZK}}$. This requires communication of $O(m \log q)$ bits. To optimize the communication cost, we can define a multivariate polynomial $f(\mathbf{x}, \mathbf{y}, z) = \mathbf{x}^\top \cdot \mathbf{y} - z$, and then prove knowledge of $\mathbf{x}, \mathbf{y}, z$ such that $f(\mathbf{x}, \mathbf{y}, z) = 0$ using the latest ZK protocol [54]. This optimization can reduce the communication cost to only $O(k \log q)$ bits, independent of m .

6.2 Support Fixed-Point and Floating-Point

There are many non-linear operations in typical ML algorithms, including ReLU, Max Pooling, Sigmoid, SoftMax, etc. These operations are complicated to compute, and may often cause some accuracy loss when values are represented as fixed-point numbers. In our implementation, we support native IEEE-754 single-precision number in ZK proofs so that we can obtain maximum accuracy.

Encoding signed, fixed-point numbers. Linear layers and non-linear layers appear alternately. It is crucial to encode data in $\mathbb{F}_p$ for linear layers so that we can enjoy our highly efficient matrix-multiplication protocol described as above. For non-linear layers, data is encoded as floating-point numbers. To eventually convert between floating-point numbers and elements over $\mathbb{F}_p$, we need to find a way to encode signed, fixed-point numbers into $\mathbb{F}_p$, and execute an encoding procedure in another direction.

Given a prime $p > 2$, we can define an encoding procedure from $\mathbb{Z}$ to $\mathbb{F}_p$ as $\text{Encode}(x \in \mathbb{Z}) = (x \bmod p)$, where an integer lies in $[-(p-1)/2, (p-1)/2]$. Note that field elements over $\mathbb{F}_p$ are represented in $[0, p-1]$. The corresponding decoding procedure is described as follows:

$$\text{Decode}(x \in \mathbb{F}_p) = \begin{cases} x, & x \leq (p-1)/2 \\ x-p, & x > (p-1)/2 \end{cases}$$

Now given a fixed-point number x and a precision parameter $s \in \mathbb{N}$, we can encode x into $\mathbb{F}_p$ by $\text{Encode}(\lfloor 2^s \cdot x \rfloor)$. If $\lfloor 2^s \cdot x \rfloor \in [-(p-1)/2, (p-1)/2]$, there is almost no accuracy loss. Encoding in another direction from elements over $\mathbb{F}_p$ to fixed-point numbers can be executed in a straightforward inverse process. There is one caveat: since we will perform matrix multiplication after non-linear layers, it is important to leave enough slack so that the precision does not overflow. In our implementation, we use a Mersenne prime $p = 2^{61} - 1$ and encode fixed-point numbers into a 30-bit range (where $s = 16$). Since in our application, the numbers never reach close to the 30-bit range, this ensures that the matrix multiplication would not overflow.

Converting between floating-point and fixed-point numbers. With values encoded as fixed-point numbers, we could convert between these fixed-point numbers and their binary representation via the arithmetic-Boolean conversion as shown in Section 4 and the encoding procedure described as above. Thus, the remaining task is to design efficient Boolean circuits for conversions between fixed-point and floating-point numbers. We use the single-precision circuits in EMP [51], where the operations conform with the IEEE-754 standard. To perform the conversion from a floating-point number to a fixed-point number, we follow the definition of IEEE-754. The key components are private logical left shift and right shift, each of which is implemented using a $(n \log n)$ -sized circuit when shifting n bits. This procedure takes about 580

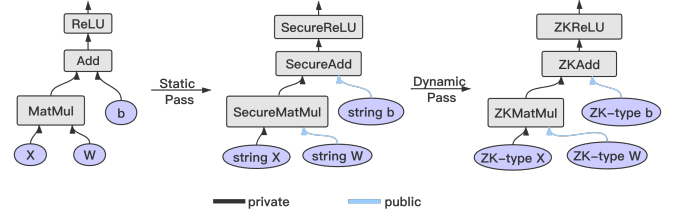


Figure 11: **Integration with TensorFlow.** Static and dynamic passes used in Rosetta to connect TensorFlow and our ZK protocol.

AND gates for $n = 61$. However, we found that converting from fixed-point to floating-point numbers is about $3 \times$ slower, since private logical right shift is done with $n = 61$, but private logical left shift is handled with $n = 24$ (defined by IEEE-754). To close the efficiency gap between two directions, we can let the prover provide a converted floating-point number as an extended witness on-demand, and then only prove in zero-knowledge the conversion from a floating-point number to a fixed-point number.

6.3 Integrating with TensorFlow

We integrate the algorithms into Rosetta [22], which is an efficient and easy-to-use framework based on TensorFlow [1]. Specifically, we implemented our ZK backend protocol in C++ to maintain high efficiency and integrated to the back-end of TensorFlow. In this way, developers could use simple interfaces in the frontend (in Python) to build complicated machine learning models without knowing details of the underlying cryptographic protocols. As a result, one can reuse the original code and interfaces of TensorFlow, and import an additional package to enable our ZK protocol. Below we discuss details of our integration. The main components of Rosetta are *static* and *dynamic passes* described as follows.

Static pass. In the frontend of TensorFlow, developers could write a model with Python language. The underlying compiler will convert the model into a graph, which consists of nodes and edges. Nodes are actually different operators, and edges are inputs/outputs of operators with specific data types (e.g., int and float). Static pass, described in Figure 11, is implemented in our framework, which additionally turns this graph into an abstract secure graph. Secure graph differs from the original graph in edges and nodes. Particularly, all the edges in secure graph are string type, which will contain the input and output information of each operator implemented with the underlying protocol (e.g., authenticated values in our ZK protocol). This is designed to be applicable to various cryptographic algorithms or protocols. Secure operators additionally specify the edges to be either public or private according to applications. The nodes in secure graph represent secure operators as shown in Figure 11.

	50 Mbps	200 Mbps	500 Mbps	1 Gbps
Conversions				
A2B	107 μs	45 μs	34 μs	29 μs
B2A	109 μs	49 μs	38 μs	33 μs
C2A	56 μs	55 μs	55 μs	55 μs
Fix2Float	50 μs	46 μs	46 μs	46 μs
Float2Fix	49 μs	46 μs	46 μs	46 μs
Machine Learning (ML) Functions				
Sigmoid	2.1 ms	1.6 ms	1.6 ms	1.6 ms
Max Pooling	1.6 ms	0.5 ms	0.4 ms	0.4 ms
ReLU	908 μs	262 μs	185 μs	188 μs
SoftMax-10	209 ms	157 ms	161 ms	171 ms
Batch Norm	415 ms	261 ms	257 ms	269 ms
Matrix Multiplications				
MatMult-512	361 ms	186 ms	185 ms	185 ms
MatMult-1024	2.42 s	1.48 s	1.39 s	1.37 s
MatMult-2048	15.19 s	11.30 s	10.63 s	10.39 s

Table 2: **Performance of the basic building blocks.** The dimension of Max Pooling is 2×2 . The dimension of Batch Normalization is $[1, 16, 16, 4]$, which stands for the batch size, height, weight and channels. For ML functions, the inputs and outputs are authenticated values in $\mathbb{F}_p$ with $p = 2^{61} - 1$. The performance result assumes that the inputs and outputs are all private to the verifier.

Dynamic pass. The graph will be executed by TensorFlow in the backend when data is fed, and the string-type data will flow across the graph. Dynamic pass shown in Figure 11 is designed to integrate the graph execution with our ZK protocol. When handling a specific operator (e.g., matrix multiplication), dynamic pass will first convert the string-type data into ZK-friendly authenticated values (i.e., ZK type in Figure 11), and then call the underlying ZK protocol for this operator and get the authenticated output. Finally, dynamic pass will convert the resulting authenticated values back to string-type data, such that the data can be handled by TensorFlow and passed to the next operator. The universal composability of our protocol ensures that our approach is secure. To make sure all operators can be composed together directly as well as reduce the memory overhead, we encode the inputs and outputs of all operators into authenticated values over $\mathbb{F}_p$.

Extendibility. In addition to our ZK protocols, Rosetta [22] is also capable of integrating with other cryptographic protocols and algorithms, such as MPC and homomorphic encryption. It is feasible to support mixed protocols between ZK proofs and MPC, where we will leave as a future work.

7 Performance Evaluation

In this section, we benchmark the speed of Mystique and how it performs on large-scale ML-inspired applications. We used three neural network models: LeNet-5² (5 layers, 62000 model parameters), ResNet-50 (50 layers, 23.5 million model parameters), and ResNet-101 (101 layers, 42.5 million model parameters). All experimental results are obtained by running the protocol over two Amazon EC2 machines of type m5.2xlarge, each with 32 GB memory. We use all CPU resources but only a fraction of the memory. The largest example is for ResNet-101 that uses 12 GB of memory. Our implementations use the latest sVOLE-based protocol [54] as the underlying ZK proof. All our implementations achieve the computational security parameter $\lambda = 128$ and statistical security parameter $\rho \geq 40$.

7.1 Benchmarking Our Building Blocks

We test the performance of our key building blocks discussed in this paper and summarized the results in Table 2. From this table, we can see that our protocol is highly scalable and all basic operations are highly efficient. The arithmetic-Boolean conversion (i.e., A2B and B2A) consists of two phases. In the preprocessing phase, two parties generate random zk-edaBits, and the execution time per zk-edaBit decreases from 95 μs to 19 μs when the bandwidth increases from 50 Mbps to 1 Gbps. In the online phase, two parties can convert authenticated wire values between arithmetic and Boolean circuits cheaply. The efficiency of the conversion from a public commitment to privately authenticated values (i.e., C2A) is mainly dominated by the computation of PRF in a Boolean circuit. It only takes around 56 μs to apply the PRF to a 64-bit data block, when the network bandwidth is at least 50 Mbps, due to the high communication efficiency of our protocol. The terms Fix2Float and Float2Fix represent the conversions between fixed-point and floating-point numbers, where the execution time for both conversions is around 46 μs per conversion when the network bandwidth is larger than 50 Mbps.

For the ZK proof of matrix multiplication (i.e., MatMul), our protocol can obtain around 185 ms of execution time for dimension 512×512 , when the network bandwidth is at least 200 Mbps. The execution time is increased to about 1.5 s and 11 s for dimensions 1024×1024 and 2048×2048 , respectively. The main efficiency bottleneck is the local computation of matrix multiplication by the prover. Compared to the state-of-the-art ZK proof for matrix multiplication [54], which takes 10 seconds to prove a 1024×1024 matrix multiplication over a network bandwidth of 500 Mbps, our ZK protocol achieves a $7\times$ improvement.

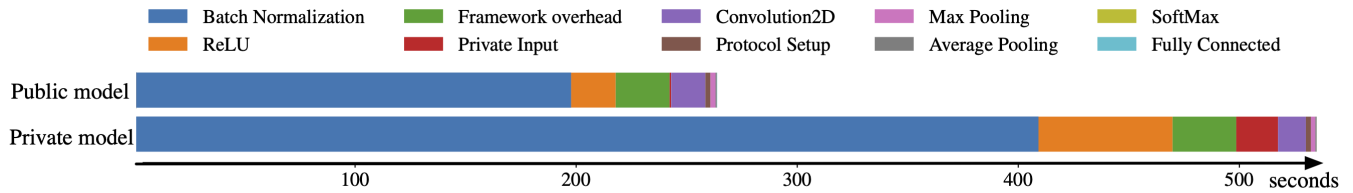


Figure 12: **Execution-time decomposition for ResNet-101 Inference.** The top bar is for public-model private-feature inference; the bottom bar is for private-model private-feature inference. The network bandwidth is throttled to 200 Mbps.

Model	Image	LeNet-5	ResNet-50	ResNet-101
Communication				
Private	Private	16.5 MB	1.27 GB	1.98 GB
Private	Public	16.5 MB	1.27 GB	1.98 GB
Public	Private	16.4 MB	0.53 GB	0.99 GB
Execution time (seconds) in a 50 Mbps network				
Private	Private	7.3	465	736
Private	Public	7.5	463	735
Public	Private	6.5	210	369
Execution time (seconds) in a 200 Mbps network				
Private	Private	5.9	333	535
Private	Public	5.5	336	541
Public	Private	4.9	158	262

Table 3: **Performance of zero-knowledge neural-network inference.** All models are trained using the CIFAR-10 dataset.

7.2 Benchmarking Private Inference

With these building blocks, we connect them together to build a ZK system to prove the inference of large neural networks as we described in Section 6.3. We consider three canonical settings, where the model parameters and model feature input can either be private to the prover or public to both parties. We focus on three neural networks: LeNet-5, ResNet-50, and ResNet-101. While the first example is relatively simple, the last two examples represent the state-of-the-art neural networks in terms of accuracy and complexity.

In Table 3, we summarize the performance for all neural networks, where the commitment on a model or data is not involved. After all optimizations, the slowest component in our protocol is Batch Normalization, which only exists in ResNet-50 and ResNet-101. For all models, we observe that when the model is private, the overall execution time is higher than the case in which the model parameters are public. This is because more operations have to be done in ZK proofs for private models. Regardless of this setting, LeNet-5 inference takes several seconds to finish. For all settings, ResNet-50

²We use ReLU as activation function instead of tanh for better accuracy.

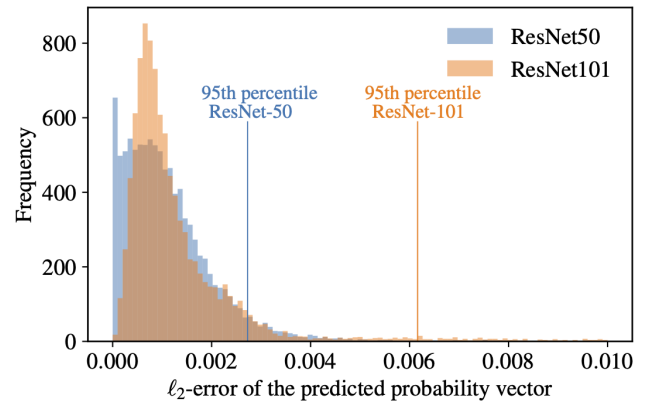


Figure 13: **ℓ_2 -norm distance between the plaintext-inference probability vector and the ZK-inference probability vector.** The mean difference is 0.0011 for ResNet-50 and 0.0019 for ResNet-101.

(resp., ResNet-101) takes about 2.6–5.6 (resp., 4.4–9) minutes to accomplish under a 200 Mbps network.

Microbenchmark. Figure 12 reports the microbenchmark of our ResNet-101 inference. We collect the time usage of different components including the protocol setup, private input (i.e., computing corresponding authenticated values), different operators and framework overhead. Significant amount of costs are used in Batch Normalization, ReLU, convolution2D and framework overhead. When the model is private, an additional proportion of time will also be used for private input. Note that the Batch Normalization takes around 70% of time in both cases because it involves complicated arithmetic operations and conversions between floating-point and fixed-point numbers, which are costly to maintain accuracy. It will be an interesting future work to further improve the efficiency of Batch Normalization and ReLU without losing accuracy.

Benchmarking the accuracy. Our approach is highly accurate, but could still cause some accuracy loss. This could particularly be a concern for deep neural networks with hundred of layers where the error could propagate and get amplified. To benchmark the accuracy of our protocol, we ran the whole CIFAR-10 testing dataset [42] containing 10000 images. CIFAR is one of the standard ML dataset to benchmark the

ML applications	LeNet-5	ResNet-50	ResNet-101
ZK for evasion attacks	9.8 <i>s</i>	316 <i>s</i>	524 <i>s</i>
ZK for genuine inference	7.2 <i>s</i>	16.4 <i>m</i>	28 <i>m</i>
ZK for private benchmark	8.2 <i>m</i>	4.4 <i>h</i>	7.3 <i>h</i>

Table 4: **Efficiency of our ZK system in different applications.** All execution time is reported based on a 200 Mbps network and two m5.2xlarge machines.

performance of algorithms. Images in CIFAR-10 are all labeled within 10 different classes, each image is a 32×32 color picture. The accuracy difference between the plaintext model and our ZK model is only 0.02% for both ResNet-50 and ResNet-101. To further understand the accuracy difference, we also compare the underlying probability vector predicted for each testing image. The dataset CIFAR-10 has 10 classes, and thus each inference produces a probability vector of length 10, denoted as $\mathbf{p}_i$ for all $i \in [1, 10000]$. The final prediction of the i -th testing image is $\text{ArgMax}_i(\mathbf{p}_i)$. We are interested in the distribution of $\|\mathbf{p}_i - \mathbf{p}'_i\|_2$, where $\mathbf{p}_i$ is from plaintext inference and $\mathbf{p}'_i$ is from ZK inference. In Figure 13, we show the ℓ_2 -norm differences of all 10000 inferences, and we can see that even for ResNet-101, the ℓ_2 -norm difference is smaller than 0.006 for 95% of the case. For LeNet-5, 99.9% of the ℓ_2 -norm difference are below 0.006. Therefore, for top- k accuracy such as $k = 5$ (commonly used for ImageNet), our ZK inference will be highly accurate.

7.3 End-to-End Applications

By connecting the private models/features to publicly committed models/features, Mystique can be used to build the three end-to-end applications mentioned in the Introduction. Since we use CIFAR-10 dataset, each image is of size 32×32 pixels and each pixel uses 3 bytes to represent the color. This means that one image is of size 3072 bytes and takes about 2.6 milliseconds to convert from publicly committed values to privately authenticated values. The sizes of three models considered in this paper are 0.25 MB, 94 MB, and 170 MB. They take 1.7 seconds, 646 seconds, and 1169 seconds to convert from a public commitment to authenticated values that can be used in our protocols directly. The cost to “pull” a publicly committed model to be used in ZK proofs is high, but it could always be amortized over multiple private inferences.

- **ZK proofs for evasion attacks.** In this case, we need to prove knowledge of two almost identical inputs that get classified to different results under a public model. Therefore, the main cost is to prove the classification result in zero-knowledge under a public model twice.
- **ZK proofs for genuine inference.** In this application, the model parameters are private but publicly committed, while the input data is public. The main overhead is from: 1)

proving the consistency between committed values and authenticated values for all model parameters; and 2) proving correct classification with private model and public input.

- **ZK proofs for private benchmark.** In this application, the testing data set is publicly committed and the model is public. Therefore, the main overhead comes from: 1) proving the consistency between committed testing data and authenticated data; and 2) proving correct classification with private input data and public model. In our example, we assume a testing data set of 100 images, and thus the second step is executed for 100 times, once for each image.

The execution time for every end-to-end application is reported in Table 4. Note that in the “ZK for private benchmark” application, 100 testing images were publicly committed, and then are converted to privately authenticated values using our conversion protocol shown in Section 5. Thus, the execution time for this application is significantly higher.

8 Conclusion

This paper presents various conversion protocols and builds zero-knowledge machine-learning inference on top of it. Although we have made a huge progress in proving ML algorithms in zero-knowledge, there are still limitations to our ZK system that deserves further exploration in future works. In particular, our ZK protocol can only prove to one verifier at a time, and the communication cost is fairly high compared to succinct ZK proofs like zk-SNARKs. We also observed a very high overhead for Batch Normalization, which may potentially be further optimized.

Acknowledgements

We thank Yuanfeng Chen, Gaofeng Huang, Junjie Shi, and Yilin Yan from MatrixElements Technologies for helping to integrate our protocol with Rosetta. Work of Kang Yang is supported by the National Key Research and Development Program of China (Grant No. 2020YFA0309703), and the National Natural Science Foundation of China (Grant Nos. 62022018, 61932019). Work of Jonathan Katz is supported by DARPA under Contract No. HR00112020025 and by NSF award #1563722. Work of Xiao Wang is supported in part by DARPA under Contract No. HR001120C0087. The views, opinions, and/or findings expressed are those of the author(s) and should not be interpreted as reflecting the position or policy of the the Department of Defense or the U.S. Government, and no official endorsement should be inferred.

References

- [1] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado,

- Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from <http://tensorflow.org>.
- [2] Martin R. Albrecht, Christian Rechberger, Thomas Schneider, Tyge Tiessen, and Michael Zohner. Ciphers for MPC and FHE. In *Advances in Cryptology—Eurocrypt 2015, Part I*, volume 9056 of *LNCS*, pages 430–454. Springer, 2015.
 - [3] Abdelrahman Aly, Emmanuela Orsini, Dragos Rotaru, Nigel P. Smart, and Tim Wood. Zaphod: Efficiently combining lss and garbled circuits in scale. In *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC’19, page 33–44, 2019.
 - [4] Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Liger: Lightweight sublinear arguments without a trusted setup. In *ACM Conf. on Computer and Communications Security (CCS) 2017*, pages 2087–2104. ACM Press, 2017.
 - [5] Toshinori Araki, Assi Barak, Jun Furukawa, Tamar Lichter, Yehuda Lindell, Ariel Nof, Kazuma Ohara, Adi Watzman, and Or Weinstein. Optimized honest-majority MPC for malicious adversaries - breaking the 1 billion-gate per second barrier. In *IEEE Symp. Security and Privacy 2017*, pages 843–862. IEEE, 2017.
 - [6] Marshall Ball, Tal Malkin, and Mike Rosulek. Garbling gadgets for Boolean and arithmetic circuits. In *ACM Conf. on Computer and Communications Security (CCS) 2016*, pages 565–577. ACM Press, 2016.
 - [7] Carsten Baum, Alex J. Malozemoff, Marc Rosen, and Peter Scholl. Mac’n’cheese: Zero-knowledge proofs for arithmetic circuits with nested disjunctions. Cryptology ePrint Archive, Report 2020/1410, 2020. <https://eprint.iacr.org/2020/1410>.
 - [8] Carsten Baum and Ariel Nof. Concretely-efficient zero-knowledge arguments for arithmetic circuits and their application to lattice-based cryptography. In *Intl. Conference on Theory and Practice of Public Key Cryptography 2020, Part I*, *LNCS*, pages 495–526. Springer, 2020.
 - [9] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable zero knowledge with no trusted setup. In *Advances in Cryptology—Crypto 2019, Part III*, volume 11694 of *LNCS*, pages 701–732. Springer, 2019.
 - [10] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. In *Advances in Cryptology—Eurocrypt 2019, Part I*, volume 11476 of *LNCS*, pages 103–128. Springer, 2019.
 - [11] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. Scalable zero knowledge via cycles of elliptic curves. In *Advances in Cryptology—Crypto 2014, Part II*, volume 8617 of *LNCS*, pages 276–294. Springer, 2014.
 - [12] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. Succinct non-interactive zero knowledge for a von neumann architecture. In *USENIX Security Symposium 2014*, pages 781–796. USENIX Association, 2014.
 - [13] Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. In *Advances in Cryptology—Eurocrypt 2011*, volume 6632 of *LNCS*, pages 169–188. Springer, 2011.
 - [14] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. In *ITCS 2012*, pages 326–349, Cambridge, MA, USA, January 8–10, 2012. Association for Computing Machinery.
 - [15] Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. Halo infinite: Recursive zk-snarks from any additive polynomial commitment scheme. Cryptology ePrint Archive, Report 2020/1536, 2020. <https://eprint.iacr.org/2020/1536>.
 - [16] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In *Advances in Cryptology—Eurocrypt 2016, Part II*, volume 9666 of *LNCS*, pages 327–357. Springer, 2016.
 - [17] Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In *ACM Conf. on Computer and Communications Security (CCS) 2018*, pages 896–912. ACM Press, 2018.
 - [18] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round OT extension and silent non-interactive secure

- p>computation. In
- ACM Conf. on Computer and Communications Security (CCS) 2019*
- , pages 291–308. ACM Press, 2019.
- [19] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. 2020.
 - [20] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *IEEE Symp. Security and Privacy 2018*, pages 315–334. IEEE, 2018.
 - [21] Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, and Greg Zaverucha. Post-quantum zero-knowledge and signatures from symmetric-key primitives. In *ACM Conf. on Computer and Communications Security (CCS) 2017*, pages 1825–1842. ACM Press, 2017.
 - [22] Yuanfeng Chen, Gaofeng Huang, Junjie Shi, Xiang Xie, and Yilin Yan. Rosetta: A Privacy-Preserving Framework Based on TensorFlow. <https://github.com/LatticeX-Foundation/Rosetta>, 2020.
 - [23] Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. Fractal: Post-quantum and transparent recursive proofs from holography. In *Advances in Cryptology—Eurocrypt 2020, Part I*, volume 12105 of *LNCS*, pages 769–793. Springer, 2020.
 - [24] Craig Costello, Cédric Fournet, Jon Howell, Markulf Kohlweiss, Benjamin Kreuter, Michael Naehrig, Bryan Parno, and Samee Zahur. Geppetto: Versatile verifiable computation. In *IEEE Symp. Security and Privacy 2015*, pages 253–270. IEEE, 2015.
 - [25] Ivan Damgård, Daniel Escudero, Tore Kasper Frederiksen, Marcel Keller, Peter Scholl, and Nikolaj Volgushev. New primitives for actively-secure MPC over rings with applications to private machine learning. In *IEEE Symp. Security and Privacy 2019*, pages 1102–1120. IEEE, 2019.
 - [26] Ivan Damgård, Jesper Buus Nielsen, Michael Nielsen, and Samuel Ranellucci. The TinyTable protocol for 2-party secure computation, or: Gate-scrambling revisited. In *Advances in Cryptology—Crypto 2017, Part I*, volume 10401 of *LNCS*, pages 167–187. Springer, 2017.
 - [27] Samuel Dittmer, Yuval Ishai, and Rafail Ostrovsky. Line-point zero knowledge and its applications. Cryptology ePrint Archive, Report 2020/1446, 2020. <https://eprint.iacr.org/2020/1446>.
 - [28] Daniel Escudero, Satrajit Ghosh, Marcel Keller, Rahul Rachuri, and Peter Scholl. Improved primitives for MPC over mixed arithmetic-binary circuits. In *Advances in Cryptology—Crypto 2020, Part II*, *LNCS*, pages 823–852. Springer, 2020.
 - [29] Dario Fiore, Cédric Fournet, Esha Ghosh, Markulf Kohlweiss, Olga Ohrimenko, and Bryan Parno. Hash first, argue later: Adaptive verifiable computations on outsourced data. In *ACM Conf. on Computer and Communications Security (CCS) 2016*, pages 1304–1316. ACM Press, 2016.
 - [30] Tore Kasper Frederiksen, Jesper Buus Nielsen, and Claudio Orlandi. Privacy-free garbled circuits with applications to efficient zero-knowledge. In *Advances in Cryptology—Eurocrypt 2015, Part II*, volume 9057 of *LNCS*, pages 191–219. Springer, 2015.
 - [31] R. Freivalds. Probabilistic machines can use less running time. In *IFIP Congress*, 1977.
 - [32] Jun Furukawa, Yehuda Lindell, Ariel Nof, and Or Weinstein. High-throughput secure three-party computation for malicious adversaries and an honest majority. In *Advances in Cryptology—Eurocrypt 2017, Part II*, volume 10211 of *LNCS*, pages 225–255. Springer, 2017.
 - [33] Irene Giacomelli, Jesper Madsen, and Claudio Orlandi. ZKBoo: Faster zero-knowledge for Boolean circuits. In *USENIX Security Symposium 2016*, pages 1069–1083. USENIX Association, 2016.
 - [34] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. Delegating computation: interactive proofs for muggles. In *40th Annual ACM Symposium on Theory of Computing (STOC)*, pages 113–122. ACM Press, 2008.
 - [35] Carmit Hazay, Peter Scholl, and Eduardo Soria-Vazquez. Low cost constant round MPC combining BMR and oblivious transfer. In *Advances in Cryptology—Asiacrypt 2017, Part I*, *LNCS*, pages 598–628. Springer, 2017.
 - [36] David Heath and Vladimir Kolesnikov. A 2.1 KHz zero-knowledge processor with BubbleRAM. In *ACM Conf. on Computer and Communications Security (CCS) 2020*, pages 2055–2074. ACM Press, 2020.

- [37] David Heath and Vladimir Kolesnikov. Stacked garbling for disjunctive zero-knowledge proofs. In *Advances in Cryptology—Eurocrypt 2020, Part III*, volume 12107 of *LNCS*, pages 569–598. Springer, 2020.
- [38] Yuval Ishai, Eyal Kushilevitz, Rafail Ostrovsky, and Amit Sahai. Zero-knowledge from secure multiparty computation. In *39th Annual ACM Symposium on Theory of Computing (STOC)*, pages 21–30. ACM Press, 2007.
- [39] Marek Jawurek, Florian Kerschbaum, and Claudio Orlandi. Zero-knowledge using garbled circuits: how to prove non-algebraic statements efficiently. In *ACM Conf. on Computer and Communications Security (CCS) 2013*, pages 955–966. ACM Press, 2013.
- [40] Jonathan Katz, Vladimir Kolesnikov, and Xiao Wang. Improved non-interactive zero knowledge with applications to post-quantum signatures. In *ACM Conf. on Computer and Communications Security (CCS) 2018*, pages 525–537. ACM Press, 2018.
- [41] Ahmed E. Kosba, Dimitrios Papadopoulos, Charalampos Papamanthou, and Dawn Song. MIRAGE: Succinct arguments for randomized algorithms with applications to universal zk-SNARKs. In *USENIX Security Symposium 2020*, pages 2129–2146. USENIX Association, 2020.
- [42] Alex Krizhevsky, Vinod Nair, and Geoffrey Hinton. Cifar-10 (canadian institute for advanced research).
- [43] Jesper Buus Nielsen, Peter Sebastian Nordholt, Claudio Orlandi, and Sai Sheshank Burra. A new approach to practical active-secure two-party computation. In *Advances in Cryptology—Crypto 2012*, volume 7417 of *LNCS*, pages 681–700. Springer, 2012.
- [44] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *IEEE Symp. Security and Privacy 2013*, pages 238–252. IEEE, 2013.
- [45] Dragos Rotaru and Tim Wood. MARbled circuits: Mixing arithmetic and Boolean circuits with active security. In *Progress in Cryptology—Indocrypt 2019*, *LNCS*, pages 227–249. Springer, 2019.
- [46] Philipp Schoppmann, Adrià Gascón, Leonie Reichert, and Mariana Raykova. Distributed vector-OLE: Improved constructions and implementation. In *ACM Conf. on Computer and Communications Security (CCS) 2019*, pages 1055–1072. ACM Press, 2019.
- [47] Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *Advances in Cryptology—Crypto 2020, Part III*, *LNCS*, pages 704–737. Springer, 2020.
- [48] Volker Strassen. Gaussian elimination is not optimal. *Numer. Math.*, 13(4):354–356, August 1969.
- [49] Riad S. Wahby, Srinath T. V. Setty, Zuocheng Ren, Andrew J. Blumberg, and Michael Walfish. Efficient RAM and control flow in verifiable outsourced computation. In *Network and Distributed System Security Symposium (NDSS)*. The Internet Society, 2015.
- [50] Riad S. Wahby, Ioanna Tzialla, abhi shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zkSNARKs without trusted setup. In *IEEE Symp. Security and Privacy 2018*, pages 926–943. IEEE, 2018.
- [51] Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient MultiParty computation toolkit. <https://github.com/emp-toolkit>, 2016.
- [52] Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *IEEE Symp. Security and Privacy 2021*. IEEE, 2021.
- [53] Chenkai Weng, Kang Yang, Xiang Xie, Jonathan Katz, and Xiao Wang. Mystique: Efficient conversions for zero-knowledge proofs with applications to machine learning. Cryptology ePrint Archive, Report 2021/730, 2021. <https://eprint.iacr.org/2021/730>.
- [54] Kang Yang, Pratik Sarkar, Chenkai Weng, and Xiao Wang. Quicksilver: Efficient and affordable zero-knowledge proofs for circuits and polynomials over any field. In *ACM Conf. on Computer and Communications Security (CCS) 2021*. ACM Press, 2021.
- [55] Kang Yang, Chenkai Weng, Xiao Lan, Jiang Zhang, and Xiao Wang. Ferret: Fast extension for correlated OT with small communication. In *ACM Conf. on Computer and Communications Security (CCS) 2020*, pages 1607–1626. ACM Press, 2020.
- [56] Greg Zaverucha, Melissa Chase, David Derler, Steven Goldfeder, Claudio Orlandi, Sebastian Ramacher, Christian Rechberger, Daniel Slamanig, Jonathan Katz, Xiao Wang, and Vladimir Kolesnikov. Picnic. Technical report, National Institute of Standards and Technology, 2019. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/round-2-submissions>.
- [57] Jiaheng Zhang, Zhiyong Fang, Yupeng Zhang, and Dawn Song. Zero knowledge proofs for decision tree predictions and accuracy. In *ACM Conf. on Computer and Communications Security (CCS) 2020*, pages 2039–2053. ACM Press, 2020.